

Homography

1. Homography

1. A projective mapping from one plane to another with the same center of projection.

2. Solve the Homography matrix

1. Given $(x_i, y_i), (x'_i, y'_i)$ for $i = 1, \dots, n$.

2. Solve $H \in \mathbb{R}^{3 \times 3}$ such that

$$3. \lambda_i \begin{bmatrix} x'_i \\ y'_i \\ 1 \end{bmatrix} = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}, \text{ where } \mathbf{v}_i = \begin{bmatrix} x'_i \\ y'_i \\ 1 \end{bmatrix}, \mathbf{H} = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix}, \text{ and } \mathbf{u}_i = \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}.$$

4. We need at least 4 point correspondences to determine $\mathbf{H}$ since $\mathbf{H}$ has 8 degrees of freedom.

$$5. \lambda_i x'_i = h_1 x_i + h_2 y_i + h_3 \quad (1)$$

$$6. \lambda_i y'_i = h_4 x_i + h_5 y_i + h_6 \quad (2)$$

$$7. \lambda_i = h_7 x_i + h_8 y_i + h_9 \quad (3)$$

8. plugging (3) to (1) and (2):

$$9. \begin{aligned} (h_7 x_i + h_8 y_i + h_9) x'_i &= h_1 x_i + h_2 y_i + h_3 \\ (h_7 x_i + h_8 y_i + h_9) y'_i &= h_4 x_i + h_5 y_i + h_6 \end{aligned}$$

$$10. \begin{aligned} h_7 x_i x'_i + h_8 y_i x'_i + h_9 x'_i - h_1 x_i - h_2 y_i - h_3 &= 0 \\ h_7 x_i y'_i + h_8 y_i y'_i + h_9 y'_i - h_4 x_i - h_5 y_i - h_6 &= 0 \end{aligned}$$

$$11. \quad \mathbf{A}\mathbf{h} = \mathbf{0}$$

$$\begin{bmatrix} -x_1 & -y_1 & -1 & 0 & 0 & 0 & x_1 x'_1 & y_1 x'_1 & x'_1 \\ 0 & 0 & 0 & -x_1 & -y_1 & -1 & x_1 y'_1 & y_1 y'_1 & y'_1 \\ -x_2 & -y_2 & -1 & 0 & 0 & 0 & x_2 x'_2 & y_2 x'_2 & x'_2 \\ 0 & 0 & 0 & -x_2 & -y_2 & -1 & x_2 y'_2 & y_2 y'_2 & y'_2 \\ & & & & \vdots & & & & \\ -x_n & -y_n & -1 & 0 & 0 & 0 & x_n x'_n & y_n x'_n & x'_n \\ 0 & 0 & 0 & -x_n & -y_n & -1 & x_n y'_n & y_n y'_n & y'_n \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \\ h_9 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

12. $A \in \mathbb{R}^{2n \times 9}$

13. Solve H using DLT:

1. For each correspondence, create $A_i \in \mathbb{R}^{2 \times 9}$ such that $A_i h = 0$.

2. Concatenate A_i into $A \in \mathbb{R}^{2n \times 9}$.

3. Compute SVD of $A^T A$ such that $A^T A = U D U^T$.

4. Let $h \in \mathbb{R}^9$ be the last column of U .

5. Reshape $h \in \mathbb{R}^9$ into $H \in \mathbb{R}^{3 \times 3}$.

14. Direct mapping from source to target

1. Denote the set of points in the source region by $\mathbf{A}$, and those in the target region by $\mathbf{B}$.

2. Generally, the projective mapping is not onto. That is, $\mathbf{H}(\mathbf{A}) \subset \mathbf{B}$ but $\mathbf{H}(\mathbf{A}) \neq \mathbf{B}$. The gray points inside the white region in the right figure denote the points that are not in $\mathbf{H}(\mathbf{A})$.

15. Inverse mapping from target to source

1. Denote the inverse mapping by $\mathbf{H}^{-1}$. We have $\mathbf{H}^{-1}(\mathbf{B}) = \mathbf{C}$ and $\mathbf{C} \subset \mathbf{A}$.

2. This means for each point $\mathbf{b} \in \mathbf{B}$, we can always find $\mathbf{a} \in \mathbf{A}$ such that $\mathbf{H}^{-1}(\mathbf{b}) = \mathbf{a}$ or $\mathbf{b} = \mathbf{H}(\mathbf{a})$.

3.



painting



monet

4. Code: homography.py

```
# -*- coding: utf-8 -*-
"""
Created on Mon May  4 17:49:10 2026

@author: reina
"""

import numpy as np
import cv2
import time
import matplotlib.pyplot as plt

def homogenize(x):
    # x is of size (d, n)
    # d - dimension
    # n - number of points
    # Converts points from inhomogeneous to homogeneous coordinates
    return np.vstack((x, np.ones((1, x.shape[1]))))

def dehomogenize(x):
    # x is of size (d+1, n)
    # d - dimension
    # n - number of points
    # Converts points from homogeneous to inhomogeneous coordinates
    return x[:-1, :] / x[-1, :]

def test_homography(H, pts_src):
    # pts_src: (d, n): (2, n)
    # H: (d+1, d+1): (3, 3)
    pts_src_hom = homogenize(pts_src) # (d+1, n)
    pts_dst_hom = H @ pts_src_hom
    pts_dst = dehomogenize(pts_dst_hom)
    return pts_dst

# TODO 1 (15%): Implement calc_homography()
def calc_homography(pts_src, pts_dst):
    # pts_src: (d, n)
    # pts_dst: (d, n)
    n = int(pts_src.shape[1])
    A = np.zeros((2 * n, 9), dtype=np.float64) # for 4 known points, 2n x 9, n=4

    for i in range(n):
        x_src = pts_src[0, i]
        y_src = pts_src[1, i]
        x_dst = pts_dst[0, i]
        y_dst = pts_dst[1, i]
        A[2 * i, 0:3] = np.array([-x_src, -y_src, -1])
        A[2 * i, 6:9] = np.array([x_src * x_dst, y_src * x_dst, x_dst])
        A[2 * i + 1, 3:6] = np.array([-x_src, -y_src, -1])
        A[2 * i + 1, 6:9] = np.array([x_src * y_dst, y_src * y_dst, y_dst])
    u, s, vh = np.linalg.svd(A)
    H = vh[-1, :].T
```

```

H = H.reshape((3, 3))
return H

def project_img(H, img_src, img_dst):
    ht_src, wid_src, _ = img_src.shape
    ht_dst, wid_dst, _ = img_dst.shape
    print("img_src.shape", img_src.shape)
    print("img_dst.shape", img_dst.shape)
    for j in range(ht_src):
        for i in range(wid_src):
            pt = np.array([i, j, 1]) # shape (1, 3)
            proj_pt = (H @ pt.T).T
            proj_pt_ = proj_pt[:, :2] / proj_pt[:, -1]
            x = int(proj_pt_[0, 0])
            y = int(proj_pt_[0, 1])
            if y > 0 and y < ht_dst and x > 0 and x < wid_dst:
                img_dst[y, x, :] = img_src[j, i, :]

    # return img_dst

def inverse_project_img(G, img_src, img_dst, pts_dst):
    ht_src, wid_src, _ = img_src.shape
    ht_dst, wid_dst, _ = img_dst.shape
    poly = cv2.fillPoly(img_dst, [pts_dst], (0, 0, 0))
    # print(poly.shape)
    # cv2.namedWindow('poly', cv2.WINDOW_NORMAL)
    # ht_window, wid_window = int(ht_dst*0.3), int(wid_dst*0.3)
    # cv2.resizeWindow('poly', ht_window, wid_window)
    # cv2.imshow('poly', poly)
    # cv2.waitKey(0)
    # cv2.destroyAllWindows()
    for j in range(ht_dst):
        for i in range(wid_dst):
            if (poly[j, i, :] == (0, 0, 0)).all():
                pt = np.array([i, j, 1]) # shape (1, 3)
                proj_pt = (G @ pt.T).T
                proj_pt_dehom = proj_pt[:, :2] / proj_pt[:, -1]
                x = int(proj_pt_dehom[0, 0])
                y = int(proj_pt_dehom[0, 1])
                if y > 0 and y < ht_src and x > 0 and x < wid_src:
                    img_dst[j, i, :] = img_src[y, x, :]

if __name__ == "__main__":
    img_src = cv2.imread("monet.jpg")
    img_dst = cv2.imread("painting.jpg")
    ht_src, wid_src, _ = img_src.shape
    ht_dst, wid_dst, _ = img_dst.shape
    pts_src = np.array([[0, 0], [wid_src, 0], [wid_src, ht_src], [0, ht_src]]) # (n, d)
    pts_dst = np.array([[299, 515], [1776, 360], [1680, 1834], [239, 1615]]) # (n, d)
    print("source_shape:", ht_src, wid_src)
    print("destination_shape:", ht_dst, wid_dst)

    cv2.namedWindow("monet", cv2.WINDOW_NORMAL)
    ht_window = int(ht_src * 0.3)
    wid_window = int(wid_src * 0.3)
    cv2.resizeWindow("monet", wid_window, ht_window)
    cv2.imshow("monet", img_src)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    cv2.namedWindow("painting", cv2.WINDOW_NORMAL)
    ht_window = int(ht_dst * 0.3)
    wid_window = int(wid_dst * 0.3)
    cv2.resizeWindow("painting", wid_window, ht_window)
    cv2.imshow("painting", img_dst)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    H = calc_homography(pts_src.T, pts_dst.T)
    H = H / H[2, 2]
    print(H)
    print(test_homography(H, pts_src.T))

    # TODO 2 (10%): Project the source image on the frame
    # formed by v1, v2, v3, and v4 in the target image via direct mapping.
    # Display the projection result.
    project_img(H, img_src, img_dst)

    cv2.namedWindow("painting", cv2.WINDOW_NORMAL)
    ht_window = int(ht_dst * 0.3)
    wid_window = int(wid_dst * 0.3)
    cv2.resizeWindow("painting", wid_window, ht_window)
    cv2.imshow("painting", img_dst)
    cv2.imwrite("direct_projection.png", img_dst)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

```

```

G = calc_homography(pts_dst.T, pts_src.T)
G = G / G[2, 2]

# TODO 3 (20%): Project the source image on the frame
#             formed by v1, v2, v3, and v4 in the target image via inverse mapping.
# Display the projection result.
inverse_project_img(G, img_src, img_dst, pts_dst)

cv2.namedWindow("painting", cv2.WINDOW_NORMAL)
ht_window = int(ht_dst * 0.3)
wid_window = int(wid_dst * 0.3)
cv2.resizeWindow("painting", wid_window, ht_window)
cv2.imshow("painting", img_dst)
cv2.imwrite("inverse_projection.png", img_dst)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

5. Homography result:



Direct Projection



Inverse Projection

6. Code: image_stitching.py

```

# -*- coding: utf-8 -*-
"""
Created on Sat Feb 21 12:42:47 2026

@author: reina
"""

import cv2
import numpy as np

def calc_homography(pts_src, pts_dst):
    A = np.zeros((8, 9), dtype=np.float64) # for 4 known point
    # correspondences (2D-2D)

    for i in range(4):
        x_src = pts_src[i, 0]
        y_src = pts_src[i, 1]
        x_dst = pts_dst[i, 0]
        y_dst = pts_dst[i, 1]
        A[2 * i, 0:3] = np.array([-x_src, -y_src, -1])
        A[2 * i, 6:9] = np.array([x_src * x_dst, y_src * x_dst, x_dst])

```

```

        A[2 * i + 1, 3:6] = np.array([-x_src, -y_src, -1])
        A[2 * i + 1, 6:9] = np.array([x_src * y_dst, y_src * y_dst, y_dst])
    u, s, vh = np.linalg.svd(A)
    H = vh[-1, :].T
    H = H.reshape((3, 3))
    return H

if __name__ == "__main__":
    img_dst = cv2.imread("library1.jpg")
    img_src = cv2.imread("library2.jpg")
    img_dst_copy = img_dst.copy()
    img_src_copy = img_src.copy()
    ht_dst, wid_dst, _ = img_dst.shape
    ht_src, wid_src, _ = img_src.shape
    pts_dst = np.array([[300, 68], [296, 217], [454, 40], [449, 217]])
    pts_src = np.array([[69, 55], [64, 211], [226, 35], [219, 209]])
    # View key points
    for i in range(4):
        cv2.circle(img_dst_copy, (pts_dst[i, 0], pts_dst[i, 1]), 2, (0, 0, 255), 5)
        cv2.circle(img_src_copy, (pts_src[i, 0], pts_src[i, 1]), 2, (0, 0, 255), 5)

    cv2.imshow("library1", img_dst_copy)
    cv2.imwrite("library1_with_points.png", img_dst_copy)
    cv2.imshow("library2", img_src_copy)
    cv2.imwrite("library2_with_points.png", img_src_copy)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

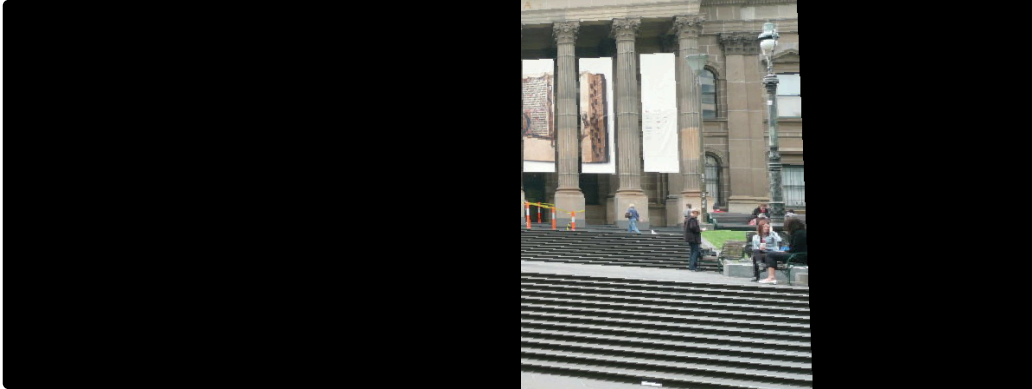
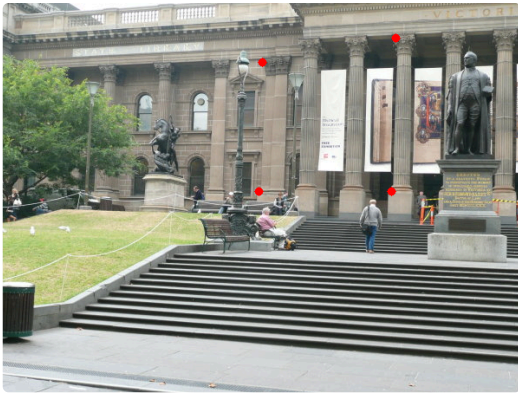
    # Compute homography for direct projection
    # H = calc_homography(pts_src, pts_dst)
    # img_canvas = np.zeros((ht_dst, wid_dst * 2, 3), dtype=np.uint8)
    # h, w, c = img_canvas.shape
    # for j in range(ht_src):
    #     for i in range(wid_src):
    #         pt = np.array([i, j, 1])
    #         proj_pt = (H @ pt.T).T
    #         proj_pt_dehom = proj_pt[:, :2] / proj_pt[:, -1]
    #         x = int(proj_pt_dehom[0, 0]); y = int(proj_pt_dehom[0, 1])
    #         if y > 0 and y < h and x > 0 and x < w:
    #             img_canvas[y, x, :] = img_src[j, i, :]

    # Compute homography for inverse projection
    G = calc_homography(pts_dst, pts_src)
    img_canvas = np.zeros((ht_dst, wid_dst * 2, 3), np.uint8)
    h, w, c = img_canvas.shape
    for j in range(ht_dst):
        for i in range(wid_dst):
            pt = np.array([i + wid_dst, j, 1])
            proj_pt = (G @ pt.T).T
            proj_pt_dehom = proj_pt[:, :2] / proj_pt[:, -1]
            x = int(proj_pt_dehom[0, 0])
            y = int(proj_pt_dehom[0, 1])
            if y > 0 and y < ht_src and x > 0 and x < wid_src:
                img_canvas[j, i + wid_dst, :] = img_src[y, x, :]

    # print(np.max(img_warped))
    cv2.imshow("img_warped", img_canvas)
    cv2.imwrite("img_warped.png", img_canvas)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    img_canvas[:, :wid_src, :] = img_dst
    cv2.imshow("img_stitched", img_canvas)
    cv2.imwrite("img_stitched.png", img_canvas)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

```



7. Code: mouse_click.py

```
# -*- coding: utf-8 -*-
"""
Created on Sat Feb 21 11:47:40 2026

@author: reina
"""

import cv2

color_red = (0, 0, 255)

def click_event(event, x, y, flags, params):
    if event == cv2.EVENT_LBUTTONDOWN:
        print(x, ' ', y)
        cv2.circle(img, (x, y), 1, color_red, 2)
        cv2.imshow('image', img)

if __name__ == '__main__':
```

```
img = cv2.imread('library1.jpg')
img_copy = img.copy()
print(img.shape)
ht, wid, _ = img.shape
is_running = True
while is_running:
    cv2.imshow('image', img)
    cv2.setMouseCallback('image', click_event)
    keyPressed = cv2.waitKey(0)
    if keyPressed == ord('q'):
        is_running = False
    if keyPressed == ord('c'):
        print('clear image')
        img = img_copy.copy()
        cv2.destroyAllWindows()
```

References:

1. DVLab, Assignment 3, National Sun Yat-sen University.